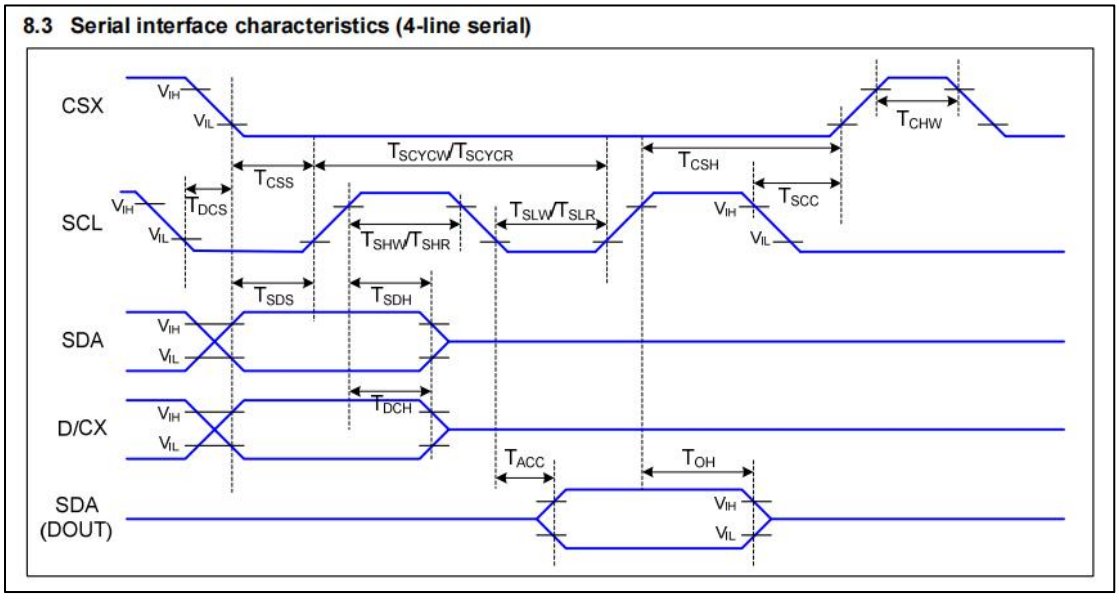
RCC SPI LCD Screen Driver Principles and Source Code Analysis

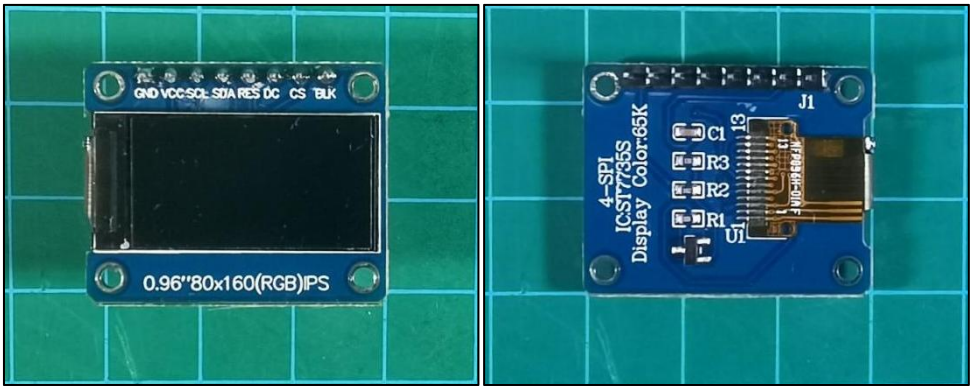
1. Principle

The ST7735 is an SPI-driven LCD screen, which sends register commands and display data to operate the LCD via SPI. The timing diagram of the ST7735S chip is shown below. From the diagram, it can be observed that: When CSX is at a high level, data transmission starts by pulling it low, simultaneously SCL is pulled low, SDA starts transmitting data, then SCK is pulled high. This sequence repeats 8 times, after which CS is pulled high, indicating the completion of data writing.



2. Physical Image of the Module

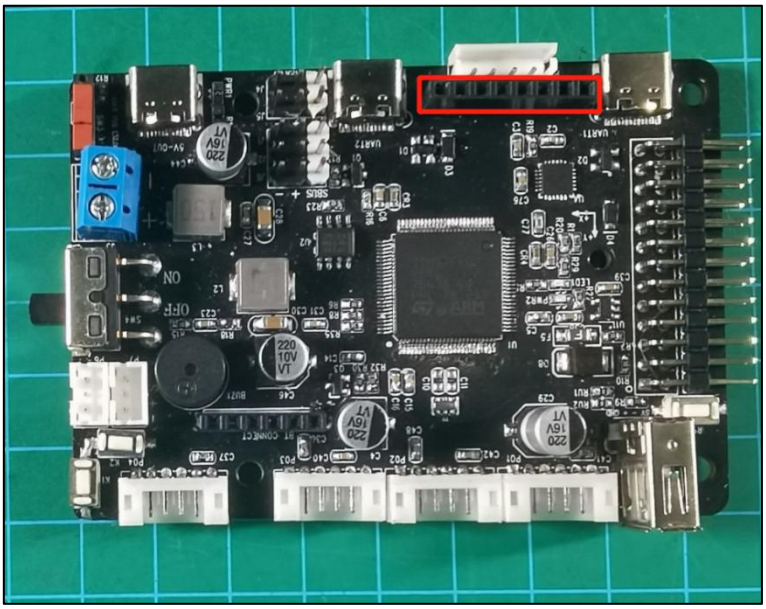
The physical images of the ST7735S LCD screen are shown in the following two pictures:



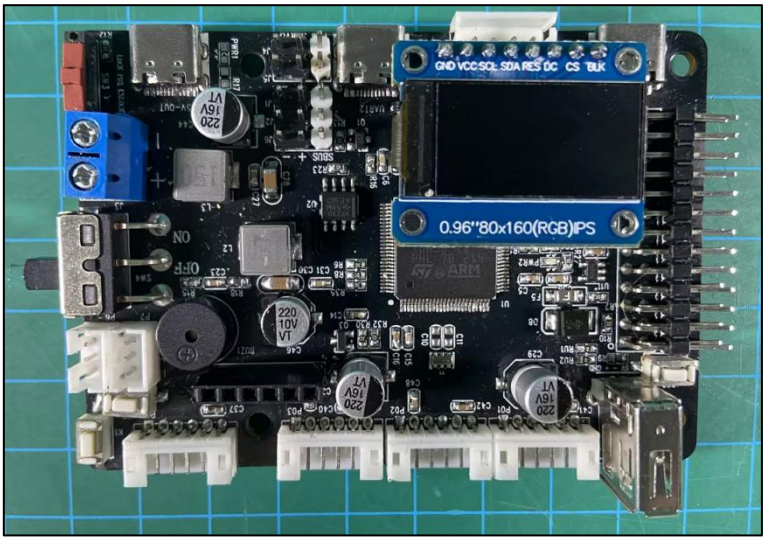
Pin connection rules (a total of 8 pins are connected to):

Wire Name	Corresponding Pin or Function
GND	GND
VCC	3.3V
SCL	SPI clock wire
SDA	SPI data writing
RES	Reset pin
DC	Command mode and data mode switching bit, low level for command mode, high level for data mode
CS	SPI Chip Select (CS)
BLK	Backlight, high level for brightness, low level for off

The wiring positions on the STM32 development board are as shown in the diagram below:



Connect the LCD module to the position indicated by the red box in the diagram above. After connecting, it should appear as shown in the diagram below:



Content Expansion:

SPI stands for Serial Peripheral Interface, which was first introduced by Motorola as

a synchronous serial transmission protocol. It is also a serial peripheral chip extension interface for microcontrollers. SPI is a high-speed, full-duplex, synchronous communication bus, allowing data to be sent and received at the same time. SPI does not define a speed limit and typically achieves speeds of up to or even exceeding 10 Mbps.

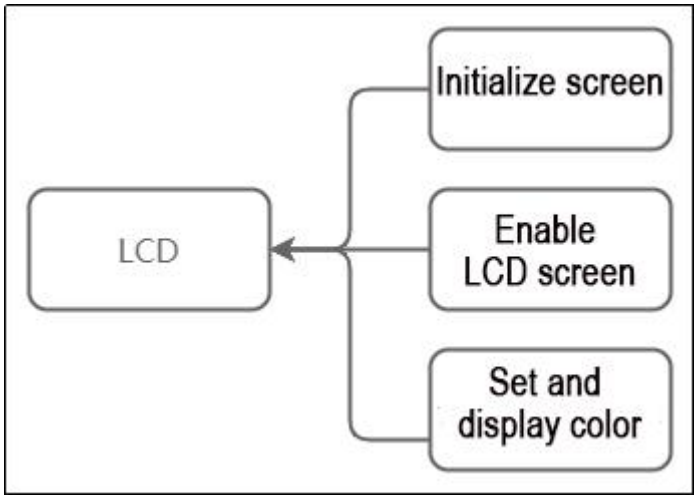
---

3. Source Code Analysis

After understanding the driving principles of the LCD screen, you can design the program structure diagram accordingly (as shown in section 2.1), and then write code for experimental testing. This section will design a program to display different colors on the LCD screen.

3.1 Program Structure

The program structure diagram is shown in the figure below:



From the diagram, it can be seen that when driving the LCD screen, it is necessary to initialize the LCD screen according to the LCD principle first, then enable the LCD screen before the program execution, and finally write functions based on the designed effects.

3.2 Program Analysis

The following content involves the program located in the folder 'RosRobotControllerM4_LCD' in the current directory, under the section titled 'RCC SPI LCD Screen Driver Example' The program can be opened and viewed using Keil5 software.

```
#include "cmsis_os2.h"
#include "lwmem_porting.h"
#include "global.h"
#include "lvgl.h"
#include "lv_port_disp.h"
#include "global.h"
#include "adc.h"

void lcds_init(void);        // LCD屏幕初始化

void app_task_entry(void *argument)
{
    // 初始化LCD屏幕参数
    lcds_init();
    // 重刷新LCD屏幕
    lcd->reset(lcd);
    for(;;) {
    // 使能LCD屏幕
    lcd->display_on(lcd);
    //  设置LCD屏幕的颜色
    lcd->set_fill_image(lcd,ST7735_CYAN);
    }
}
```

- Initialize screen

Before powering on the screen, it's necessary to initialize the parameters of the LCD screen, which are located in the "Hiwonder/System/app.c" file. The function is as shown below:

```
// 初始化LCD屏幕参数
lcds_init();
```

The image below illustrates the content of the lcd_init function:

```
void lcds_init(void)
{
    ST7735ObjectTypeDef *st7735 = LWMEM_CCM_MALLOC(sizeof(ST7735ObjectTypeDef));
    st7735_object_init(st7735);
    st7735->base.height = 80;
    st7735->base.width = 160;
    st7735->x_offset = 1;
    st7735->y_offset = 26;
    st7735->set_backlight = lcd1_set_blacklight;
    st7735->set_dc_cs = lcd1_set_dc_cs;
    st7735->set_res = lcd1_set_res;
    st7735->spi_write = lcd1_spi_write,
    st7735->sleep_ms = DelayMs;
    st7735->set_dc_cs(st7735, 1, 1);
    lcd = (DisplayObjectTypeDef*)st7735;
}
```

st7735 is an instantiation of a structure object st7735s, with the following parameters:

base.height represents the height, set to 80 pixels.

base.width represents the width, set to 160 pixels.

base.x_offset represents the offset in the x-direction.

base.y_offset represents the offset in the y-direction.

set_backlight sets the background contrast of the LCD screen.

set_dc_cs sets the data/command and chip select pins interface (ST7735ObjectTypeDef *self, uint32_t new_dc, uint32_t new_cs).

set_res similarly sets the data/command and chip select pins interface (ST7735ObjectTypeDef *self, uint32_t new_res).

sleep_ms represents a delay interface, indicating a time delay event, measured in milliseconds.

The final statement lcd = (DisplayObjectTypeDef*)st7735 represents the conversion of st7735 into DisplayObjectTypeDef, which is the base class inside the ST7735ObjectTypeDef structure. This facilitates subsequent program writing.

- **Enable the LCD screen**

The function to enable the LCD screen is display_on. For specific implementation details, refer to the "Hiwonder/Peripherals/display_st7735.c" file.

```
static void export_st7735_reset(DisplayObjectTypeDef *self_base)
{
  ST7735ObjectTypeDef *self = (ST7735ObjectTypeDef*)self_base;
    self->set_backlight(self, 100);
    st7735_hard_reset(self);
    //st7735_soft_reset(self);
    write_command(self, 0x11); //Sleep out
    self->sleep_ms(self, 50); //Delay 120ms
```

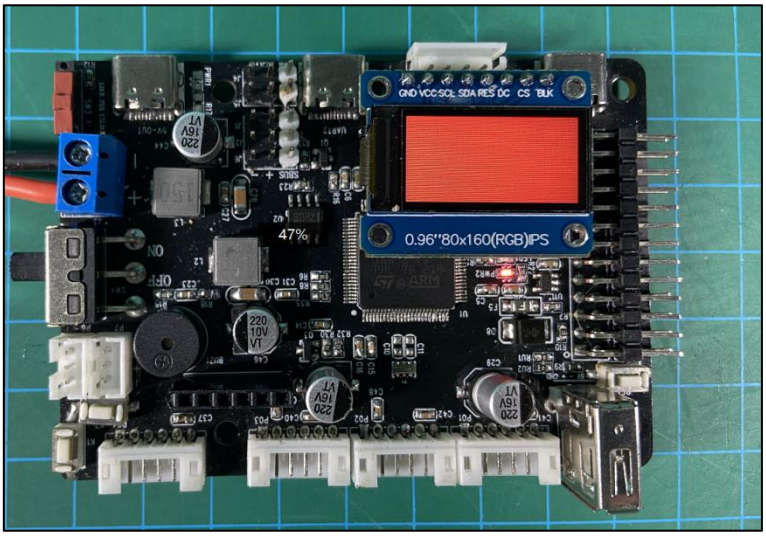
The function to enable the LCD screen is display_on. For specific implementation details, refer to the "Hiwonder/Peripherals/display_st7735.c" file.

- **Set and Display Color**

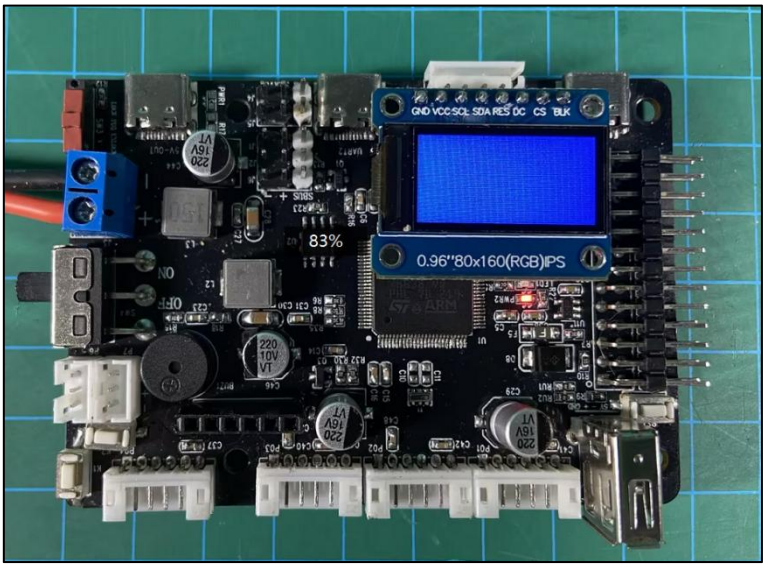
After enabling the LCD screen, you can use it to display colors. The function for displaying colors can also be found in the app.c file in the Hiwonder/System directory:

```
//  设置LCD屏幕的颜色
lcd->set_fill_image(lcd,ST7735_RED);
```

The image above shows that within the for loop, the color of the LCD screen is set to ST7735_RED (red). After programming, the effect is as shown below:



Similarly, changing the function ST7735_RED to ST7735_BLUE (blue) yields the effect shown in the image below:



For more color settings, you can refer to the display_st7735.h file in the Hiwonder/Peripherals directory (the definition section is shown below). Simply change the second parameter of the set_fill_image function.

```
#define  ST7735_BLACK   0x0000  // 黑色
#define  ST7735_BLUE    0x001F  // 蓝色
#define  ST7735_RED     0xF800  // 红色
#define  ST7735_GREEN   0x07E0  // 绿色
#define  ST7735_CYAN    0x07FF  // 蓝绿色
#define  ST7735_MAGENTA 0xF81F  // 品红色
#define  ST7735_YELLOW  0xFFE0  // 黄色
#define  ST7735_WHITE   0xFFFF  // 白色
```